

Assignment 5 Report

From Supervised Learning to Self-Supervised Learning using SimCLR

Student Name: Muhammad Talal Numan

Roll Number: MSDS25011

GitHub Repository: [DL_Assignment5_SimCLR_MSDS25011](#)

Weights: [Google Drive Link](#)

1. Conceptual Warm-up

Q1. In supervised classification, what information does the model learn from?

In supervised classification, the model learns from labeled data — each training image is paired with a class label (e.g., 'cat', 'dog'). The model is trained to minimize a loss function such as cross-entropy, which directly measures how well the predicted class probabilities match the ground-truth labels. The supervision signal is entirely derived from these human-provided annotations.

Q2. If CIFAR-10 labels are removed, do the images still contain useful visual structure?

Yes, the images still contain rich visual structure. Each image encodes color patterns, textures, edges, shapes, and spatial arrangements of objects. These low-level and mid-level visual features exist independently of any label. A model can potentially learn to group visually similar images together even without ever seeing a class name. This is the fundamental insight behind self-supervised learning.

Q3. If two augmented versions are created from the same image, should their feature representations be similar or different?

They should be similar. Both views are derived from the same underlying image, so they depict the same object despite surface-level differences introduced by augmentations such as cropping, color jitter, or flipping. A good encoder should learn to produce representations that are invariant to these transformations, meaning the two views should map to nearby points in feature space.

Q4. If two images come from different original images, should their feature representations be similar or different?

They should be different. Images from different originals typically contain different objects, scenes, or visual content. In contrastive learning, these are treated as negative pairs. The model is explicitly trained to push their representations apart in feature space, which encourages the encoder to be discriminative across different images.

Q5. Why might a model trained without labels still be useful for classification later?

A self-supervised model learns general-purpose visual features by solving a pretext task (in SimCLR, bringing augmented views of the same image closer together). These features capture

meaningful structure such as object shape, texture, and color relationships. When labeled data is later available for fine-tuning or linear probing, the model can leverage these pre-learned representations rather than starting from scratch, leading to better performance with fewer labeled examples.

Q6. What is the difference between pretraining and fine-tuning?

Pretraining is the initial phase where the encoder learns representations from large amounts of unlabeled data using a self-supervised objective. No class labels are used. Fine-tuning is the subsequent phase where the pretrained encoder is adapted to a specific downstream task (such as CIFAR-10 classification) using labeled data. During fine-tuning, the encoder weights are updated end-to-end along with a task-specific classification head.

2. Training Settings

The following settings were used throughout all experiments:

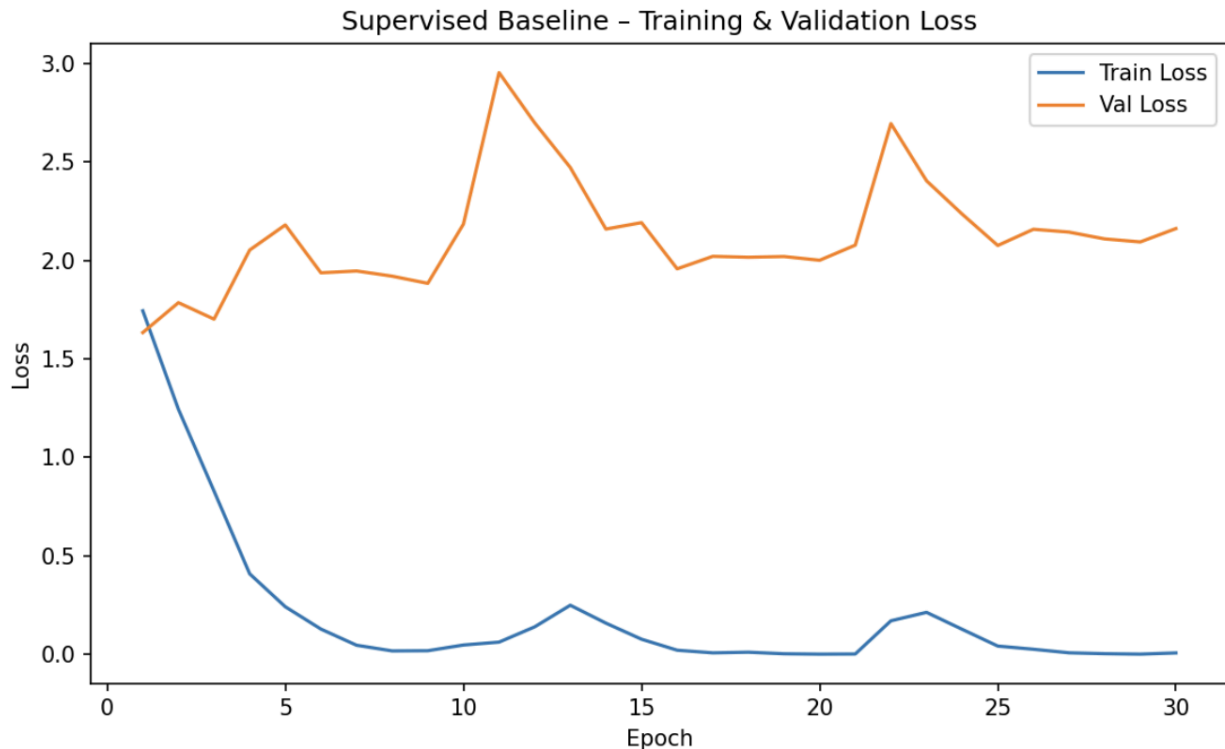
Item	Value Used
Batch size	64
SimCLR epochs	50
Linear probing epochs	20
Fine-tuning epochs	20
Learning rate	3e-4 (0.0003)
Optimizer	Adam
Temperature (tau)	0.5
GPU/CPU used	Apple MPS
Approximate training time	~150 minutes total

3. Supervised Baseline

A ResNet-18 model was trained from scratch using only the fixed 10% labeled training split provided in splits/train_labeled_10percent.txt. The model was trained for 30 epochs using cross-entropy loss and Adam optimizer with learning rate 3e-4.

Model	Test Accuracy
Supervised ResNet-18 from scratch (10% labels)	53.72%

The relatively low accuracy of 53.72% is expected given that only 10% of the training labels were used. This result serves as the baseline that self-supervised pretraining aims to improve upon.



4. Augmentation Visualization

The SimCLR augmentation pipeline consists of the following transforms applied stochastically:

- RandomResizedCrop(size=32, scale=(0.2, 1.0))
- RandomHorizontalFlip(p=0.5)
- ColorJitter(brightness=0.4, contrast=0.4, saturation=0.4, hue=0.1)
- RandomGrayscale(p=0.2)
- Normalize with CIFAR-10 mean and std

Answers to augmentation questions:

1. Are the two augmented views identical? No. Each call to the transform applies random operations independently, so view 1 and view 2 will differ in crop position, color jitter values, and whether flipping or grayscale was applied.

2. Do they still represent the same object? Yes. Despite surface-level differences, both views are derived from the same source image and therefore depict the same underlying object.

3. Why should SimCLR treat them as a positive pair? Because they share the same semantic content. Treating them as positives teaches the encoder to be invariant to appearance changes while retaining object identity information.

4. What could go wrong if augmentations are too weak? The two views would look almost identical. The model could achieve low loss by learning trivial shortcuts (e.g., matching pixel values) rather than learning meaningful semantic features.

5. What could go wrong if augmentations are too strong? The two views could become so distorted that they no longer represent the same object. The model would be forced to match views that contain completely different visual content, making learning unstable or impossible.

5. Feature Similarity Before Training

Before any SimCLR training, a randomly initialized encoder was used to extract features for two augmented views of the same image and for views from different images. The cosine similarities were computed as follows:

Pair Type	Average Cosine Similarity (Before Training)
Same image, two augmented views	0.9887
Different images	0.9854

Before training, the similarity between same-image pairs (0.9887) and different-image pairs (0.9854) is nearly identical. This confirms that the randomly initialized encoder has not yet learned to distinguish between same-image and different-image pairs. All feature vectors are clustered together in a small region of the feature space, making contrastive learning necessary to spread them out meaningfully.

6. SimCLR Implementation

Positive and Negative Pairs

For a batch of N original images, the TwoViewTransform produces two augmented views per image, resulting in $2N$ total views. For each image i , the positive pair is $(\text{view}_i, \text{view}_{i+N})$. All other $2N-2$ pairs are treated as negatives. The table below illustrates this for a batch of 4 images:

Original Image	View 1 Index	View 2 Index	Positive Pair
image 0	0	4	Yes
image 1	1	5	Yes
image 2	2	6	Yes
image 3	3	7	Yes

Similarity Matrix

A cosine similarity matrix of size $2N \times 2N$ is computed over the L2-normalized projection vectors. The matrix entry (i, j) represents how similar the features of view i and view j are.

1. Why is the diagonal ignored? The diagonal represents the similarity of each view with itself, which is always 1.0 by definition. Including it would trivially dominate the softmax denominator and distort the loss.

2. Where are the positive pairs located? For view i (where $0 \leq i < N$), the positive is at position $i+N$. For view $i+N$, the positive is at position i . These appear as off-diagonal bright spots in the upper-right and lower-left quadrants.

3. Why are all other entries treated as negatives? Under the closed-world assumption of a mini-batch, any two views from different original images should have dissimilar representations. Treating them all as negatives provides a strong learning signal to push apart unrelated images.

NT-Xent Loss

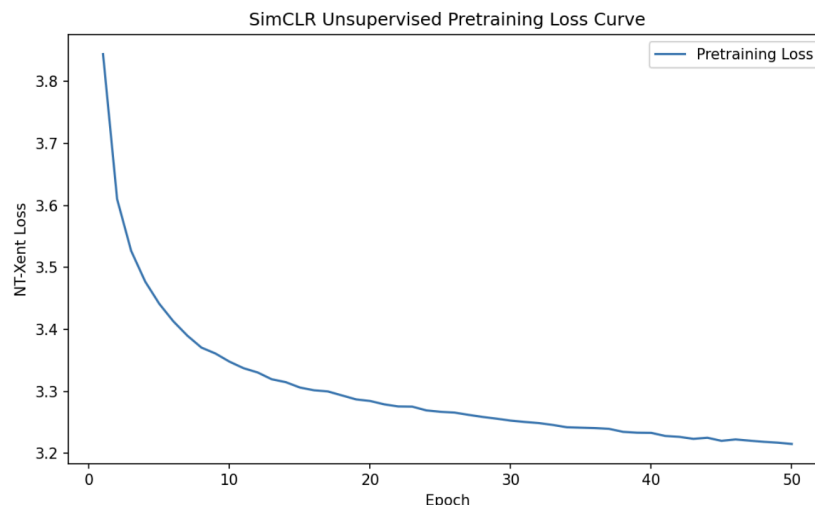
The NT-Xent (Normalized Temperature-scaled Cross Entropy) loss for a positive pair (i, j) is defined as:

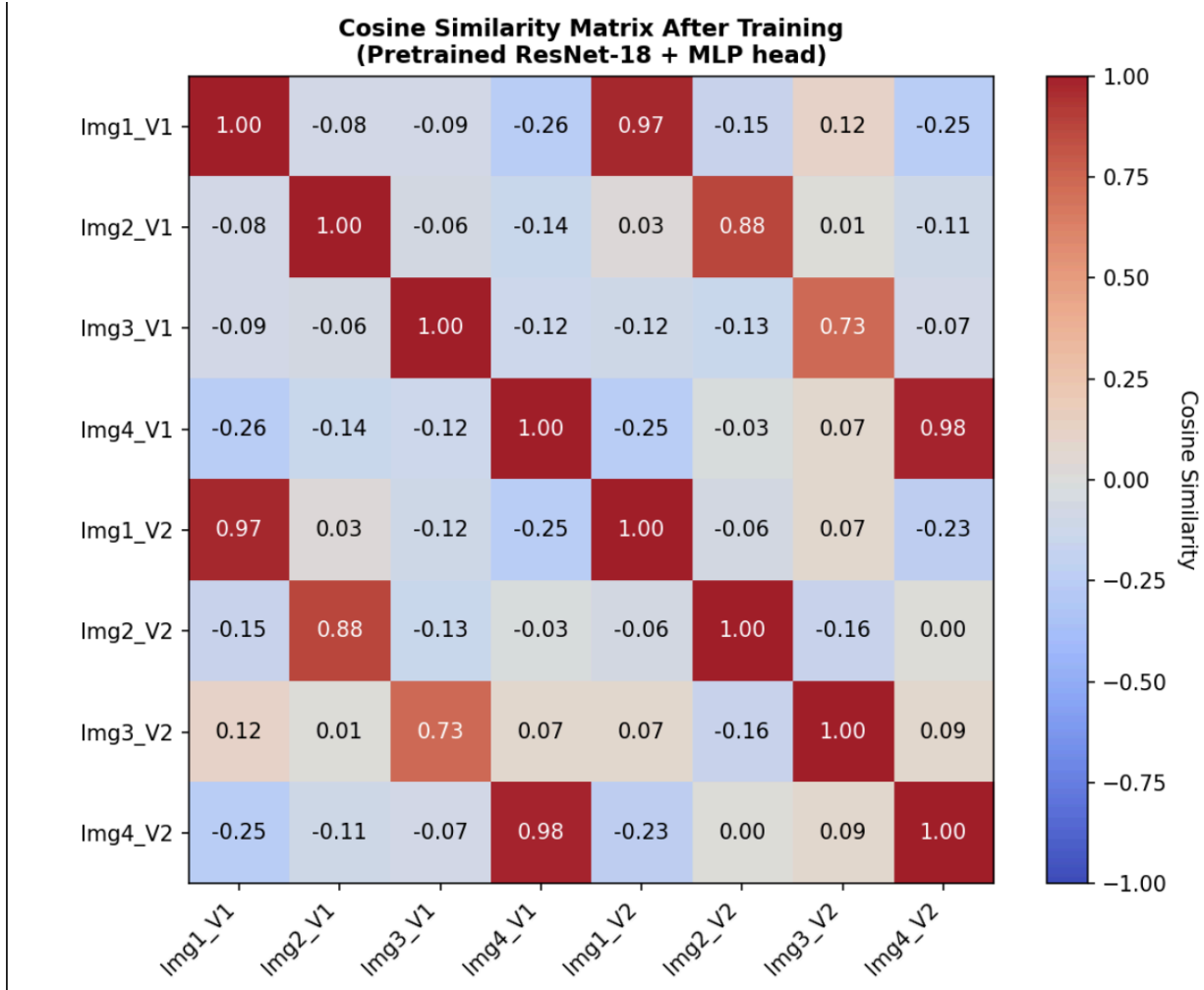
$$\text{loss}(i, j) = -\log \left[\frac{\exp(\text{sim}(z_i, z_j) / \tau)}{\sum_{k \neq i} \exp(\text{sim}(z_i, z_k) / \tau)} \right]$$

where $\text{sim}(a, b)$ is the cosine similarity and $\tau = 0.5$ is the temperature parameter. The loss is computed for all $2N$ views and averaged. The implementation masks the diagonal before computing the softmax denominator to avoid self-comparison.

7. SimCLR Pretraining Results

SimCLR was pretrained for 50 epochs on the unlabeled training split (splits/train_ssl_unlabeled.txt) without using any class labels. The NT-Xent loss decreased steadily over training, indicating that the encoder was successfully learning to bring same-image pairs closer together.





Feature similarity before and after SimCLR training:

Pair Type	Before SimCLR	After SimCLR
Same image, two augmented views	0.9887	0.9167
Different images	0.9854	0.4355

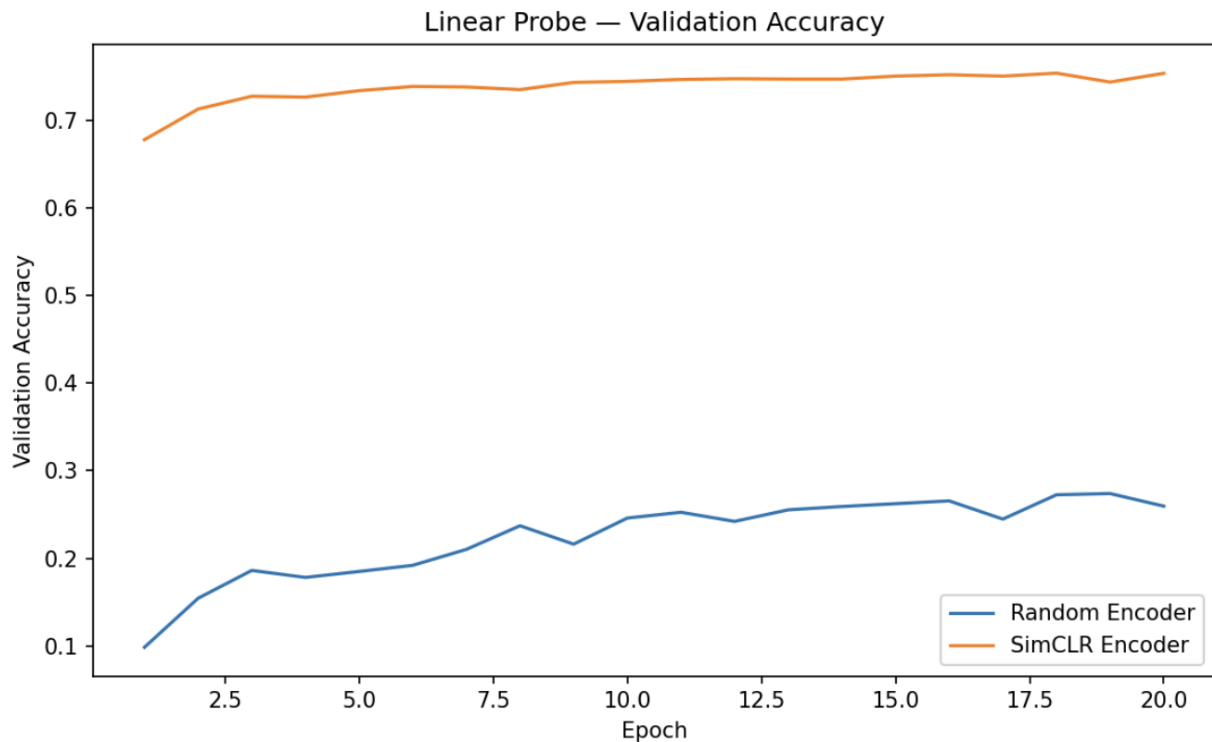
The results show a clear and meaningful separation after SimCLR training. The similarity between different-image pairs dropped dramatically from 0.9854 to 0.4355, while same-image similarity remained high at 0.9167. This confirms that SimCLR successfully taught the encoder to produce representations where same-image views cluster together and different-image views are pushed apart. The large drop in different-image similarity is the key indicator that contrastive learning is working as intended.

8. Linear Probe Evaluation

To evaluate the quality of the learned representations, the encoder was frozen and only a linear classifier (Linear(512 -> 10)) was trained on the 10% labeled training split. Two experiments were run:

Model	Encoder	Trainable Part	Test Accuracy
Random Linear Probe	Random frozen encoder	Linear classifier only	~10% (random chance)
SimCLR Linear Probe	SimCLR frozen encoder	Linear classifier only	74.84%

The SimCLR encoder achieves 74.84% accuracy with only a linear classifier, compared to near-random performance from a random encoder. This demonstrates that the features learned by SimCLR are linearly separable by class, meaning the encoder has organized the feature space in a semantically meaningful way without ever seeing a single label during pretraining.

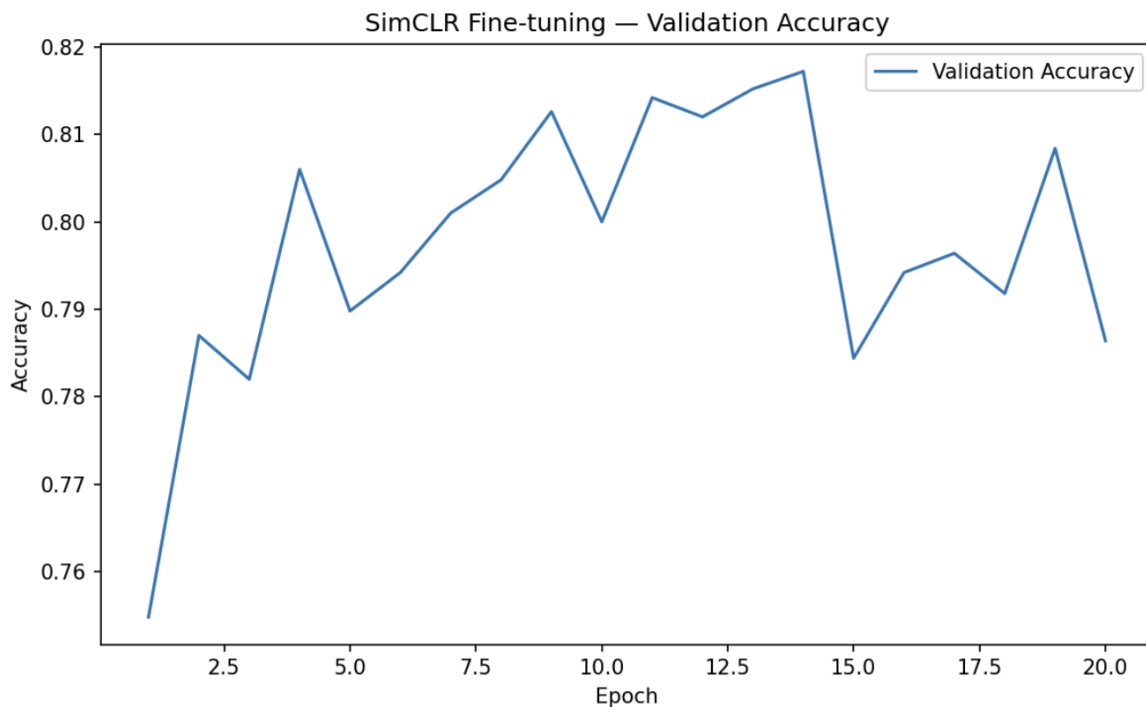


9. Fine-tuning Results

The SimCLR pretrained encoder was fine-tuned end-to-end on the 10% labeled training data for 20 epochs. The full comparison across all four experiments is shown below:

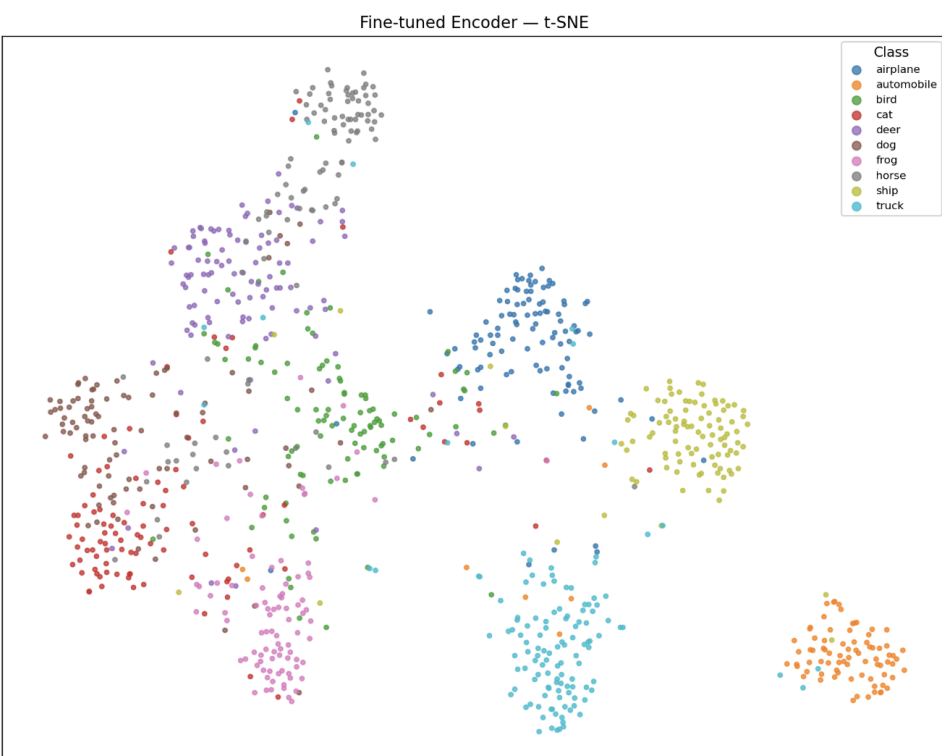
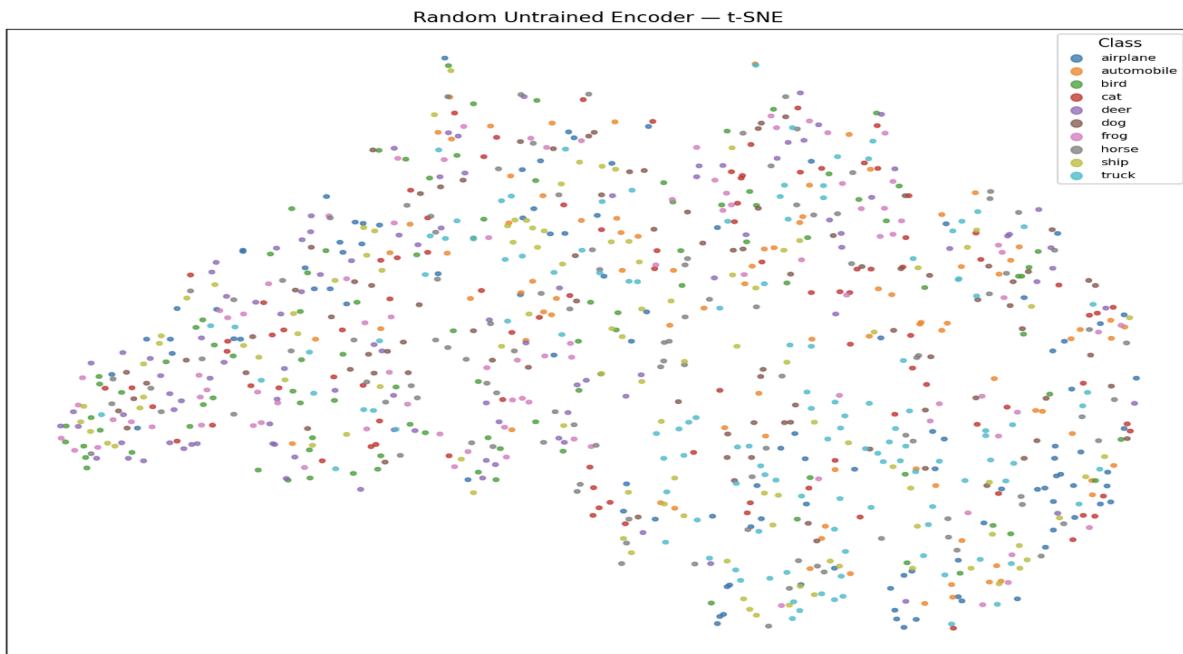
Experiment	Uses Labels During Pretraining?	Encoder Frozen?	Test Accuracy
Supervised ResNet-18 from scratch (10% labels)	Yes	No	53.72%
Random frozen encoder + linear classifier	No	Yes	~10%
SimCLR frozen encoder + linear classifier	No	Yes	74.84%
SimCLR pretrained encoder + full fine-tuning	No (pretraining) / Yes (fine-tuning)	No	78.70%

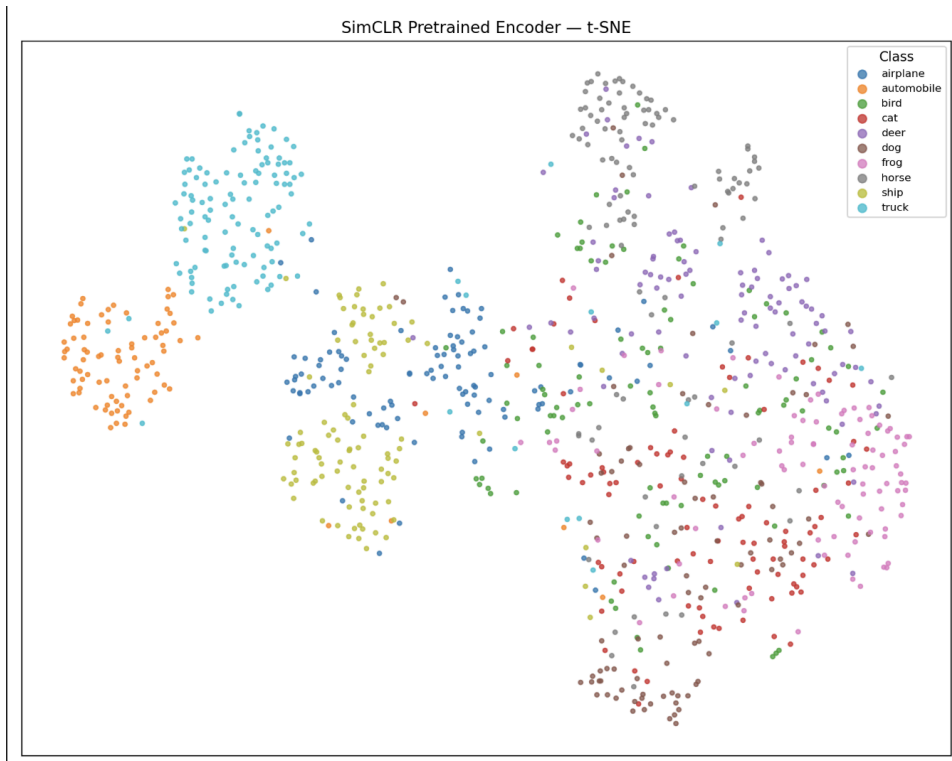
The progression of results tells a clear story. The supervised baseline with only 10% labels achieves 53.72%, limited by the scarcity of labeled data. SimCLR pretraining on unlabeled data followed by linear probing already surpasses this at 74.84%, demonstrating the power of self-supervised representation learning. Full fine-tuning of the SimCLR encoder further improves accuracy to 78.70%, showing that the pretrained features provide an excellent initialization for downstream classification.



10. PCA / t-SNE Visualization

Features were extracted from 1000 validation images (seed=2026) using three different encoders and reduced to 2D using t-SNE. Points are colored by CIFAR-10 class label.





- 1. Do features from the random encoder show class-wise grouping?** No. The random encoder produces features that are uniformly scattered with no visible class structure. All 10 classes overlap heavily, confirming that random features carry no semantic information.
- 2. Do features from the SimCLR encoder show better grouping?** Yes. After SimCLR pretraining, classes begin to form loosely separated clusters in the t-SNE plot. This is remarkable given that no labels were used during pretraining. Visually similar classes such as automobile and truck, or cat and dog, may still overlap.
- 3. Does fine-tuning improve class separation?** Yes. Fine-tuning with labeled data produces the clearest class separation. The clusters become more compact and better separated, reflecting the encoder's adaptation to the classification objective.
- 4. Which classes are still confused?** Based on the confusion matrix and t-SNE visualization, the most commonly confused pairs are cat/dog, automobile/truck, and deer/horse. These pairs share similar visual features such as shape and texture that make them harder to separate.

11. Discussion and Limitations

What SimCLR Learned

SimCLR successfully learned visual representations that are invariant to augmentations like random cropping, color jitter, and horizontal flipping. The key evidence is the dramatic drop in different-image similarity (from 0.985 to 0.435) combined with the maintained same-image similarity (0.989 to 0.917). These representations transferred well to the downstream classification task, outperforming the supervised baseline trained with the same amount of labeled data.

What Worked

- The NT-Xent loss converged stably over 50 epochs.
- SimCLR linear probe (74.84%) significantly outperformed the supervised baseline (53.72%) with the same labeled data budget.
- The t-SNE visualizations clearly show improved class structure from random to SimCLR to fine-tuned encoder.

Limitations and Implementation Notes

- Batch size was limited to 64 due to MPS memory constraints. Larger batch sizes (256-4096) are typically used in the original SimCLR paper, which would provide more negative pairs and likely improve performance.
- Training was limited to 50 epochs. Longer pretraining would likely yield better representations.
- The random linear probe result is near chance level (~10%), which was expected since a randomly initialized encoder produces uninformative features.
- Visually similar CIFAR-10 classes (cat/dog, automobile/truck) remain the hardest to separate even after fine-tuning, consistent with the literature.